

Threadlab Multithreaded AI Data Processing System

Pranav Shripannavar | PRN 202501110195
Division C | Batch C2 | Course OS | Assignment 2

Problem statement. Process a large dataset using one thread and multiple threads, divide the work explicitly, record each thread's completion, compare execution times and explain thread-management overhead.

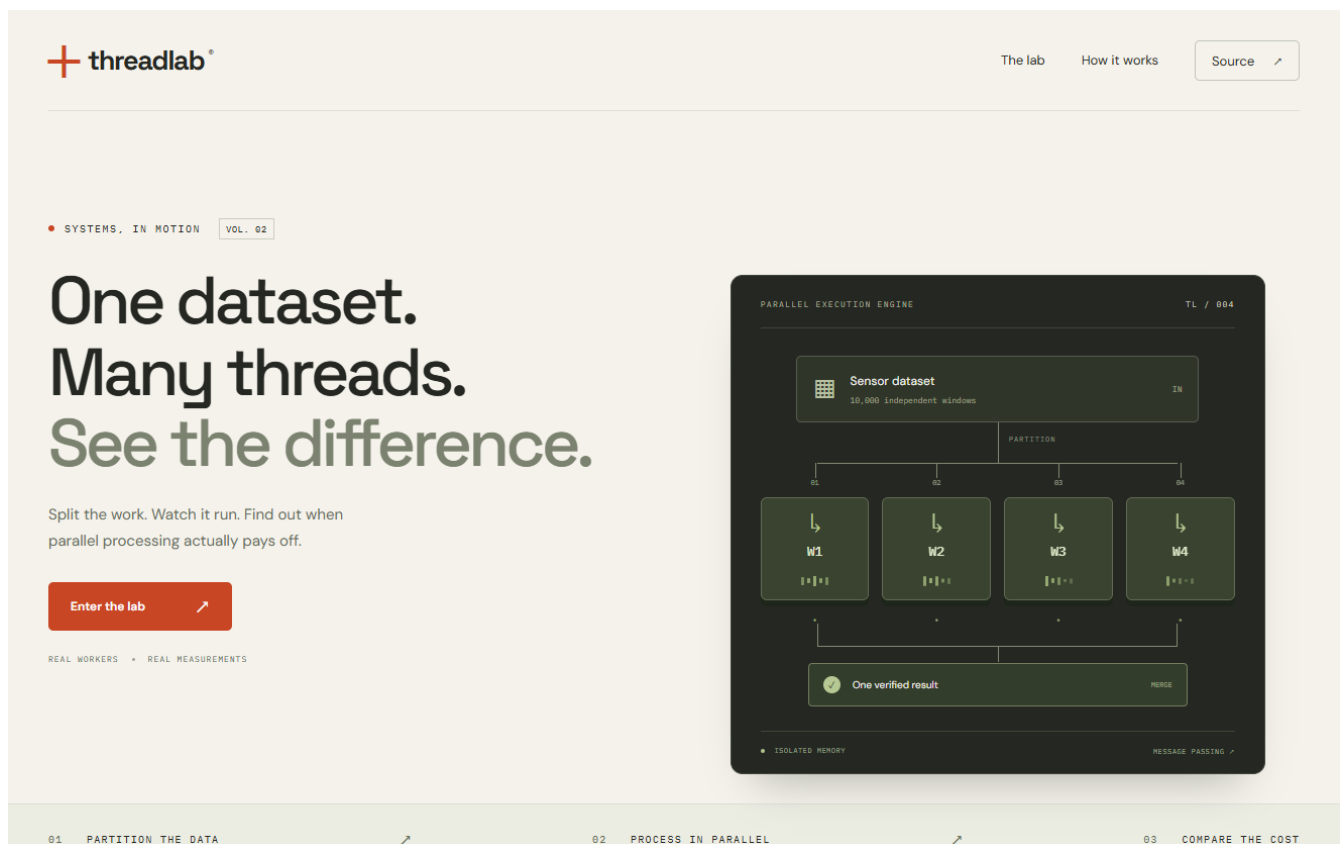


Figure 1. Deployed Threadlab interface. Charcoal, ivory and orange; real measurements and responsive controls.

Implemented scope

The application processes 10,000 sensor windows by default, with options from 1,000 to 100,000 records. Real browser Web Workers run identical spectral feature extraction in one-thread and 2/4/8-thread configurations. A server-side Gemini endpoint interprets the results. A standalone Python implementation is included for the assignment's listed programming languages.

Live: os-assignment-2-sigma.vercel.app

Source: github.com/PranavShripannavar/os-assignment-2

OS concepts and approach

Unit 2: Process and Thread Management. CO2: applying process management and synchronization techniques to system-level problems.

Concept	Implementation
Thread creation and lifecycle	Create worker, load computation module, process partition, post completion, terminate. Cancellation, errors and timeout terminate active workers.
Workload partitioning	Contiguous balanced partitions cover every record once. No gaps or overlaps; partition sizes differ by at most one.
Process vs. thread	Threads provide execution within an environment; processes own resources. Browser worker execution is real, but browser process placement and kernel scheduling policy are not inspected.
Memory and synchronization	Workers have isolated JavaScript memory. Transferable buffers move chunk ownership. Completion messages synchronize reduction; there is no shared mutable state.

Algorithm

1. Generate N records using a deterministic seed of 42; each record contains 16 sensor samples. Generation is outside the timed region.
2. Center samples around zero. Project each window onto B frequency bands using sine and cosine sums. Average spectral energy; flag values above 0.1 as anomalies.
3. Quantize each record's energy before summing. Compare record count, integer energy checksum and anomaly count between versions.
4. Run one fresh worker over all records. Then create T fresh workers. Worker i receives $\lfloor iN/T \rfloor$, $\lfloor (i+1)N/T \rfloor$.
5. Collect every completion message, reduce results and compute speedup = single wall time / multithread wall time. Report each partition and its compute time.

Timing contract

Browser wall time includes worker creation, module loading, chunk copying, transfer, computation, scheduling and receiving results. It excludes initial dataset generation and worker termination. Per-worker compute time measures only the feature-extraction call. The single-thread baseline also uses a worker to keep the interface responsive.

The baseline runs first. Cache/JIT warming, thermal state, network module loading and competing tasks can bias a single trial. New worker instances do not imply an empty HTTP cache. Repeat measurements before drawing general conclusions.

Execution evidence

Production browser run | 16 September 2026 | 10,000 records | 32 bands | 4 workers

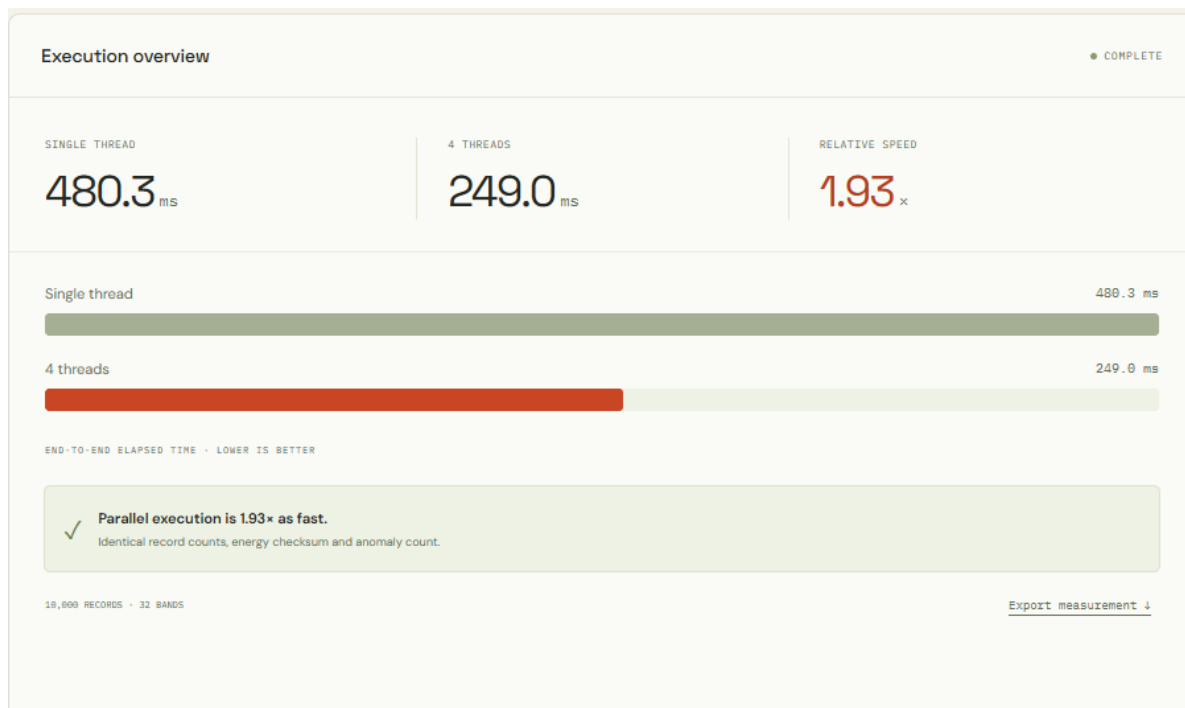


Figure 2. Actual production timing comparison; no simulated benchmark numbers.

Measurement	Single worker	Four workers
Wall time	480.3 ms	249.0 ms
Records processed	10,000	10,000
Energy checksum	831,583,345	831,583,345
Anomalies	1,830	1,830
Relative speed	1.00x	1.93x

Per-thread completion

Worker	Record range*	Processed	Compute	State
W1	1-2,500	2,500	67.1 ms	Complete
W2	2,501-5,000	2,500	66.8 ms	Complete
W3	5,001-7,500	2,500	64.6 ms	Complete
W4	7,501-10,000	2,500	65.2 ms	Complete

*UI ranges are one-based and inclusive. Exported JSON uses zero-based, half-open ranges. Source evidence: evidence/live-measurement.json. All workers produced actual completion messages.

Performance analysis

The recorded browser run achieved **1.93x speedup**: 480.3 ms became 249.0 ms, with identical outputs. Work was evenly divided into four 2,500-record partitions. The longest worker computation was 67.1 ms. Parallel wall time minus that computation was **181.9 ms**; this is non-kernel elapsed time, including loading, copying, communication and scheduling, not an isolated overhead measurement.

Why more threads can lose

A separate small-input test (1,000 records, 8 bands, 2 workers) measured 112.2 ms single-threaded and 153.9 ms multithreaded: 0.73x relative speed, or 37.2% longer. Startup and coordination can outweigh saved computation. Browser-reported logical processors do not prove actual core allocation, CPU utilization or linear scaling.

Python implementation: real threads, different runtime

Python test: 10,000 records, 32 bands	Measured result
Main-thread baseline	1112.48 ms
Four native threads	1145.85 ms
Speedup / correctness	0.97x / identical outputs
Native worker IDs	11224, 29752, 26816, 18944

Python uses ThreadPoolExecutor and a barrier to ensure all requested native threads participate. Its parallel timing includes pool creation, synchronization and shutdown. The default CPython GIL limits concurrent execution of this pure-Python CPU workload, so the measured slight slowdown is plausible. Python and browser timings are separate experiments, not equivalent runtime measurements.

Connection to AI pipelines

Independent sensor windows can be transformed concurrently before inference. Partitioning, feature extraction and reduction correspond to parallel preprocessing in ML systems. These results do not benchmark model training or GPU workloads. Gemini provides an interpretation of measured data; it does not generate timings or decide output correctness.

Backend evidence: production POST /api/analyze returned HTTP 200 and a complete Gemini response. The model selected for the screenshot run was gemini-3-flash-preview. Measurements are structurally validated, but user-device timings are not server-attested.

Tests, submission and demonstration

Test	Input / action	Observed outcome
Partition coverage	1, 7 and 10,003 records; legal thread counts	Exact coverage, balanced ranges
Determinism	Same seed and different seed	Same seed repeats; different seed changes data
Known-value computation	All samples set to 0.5	Zero energy, no anomalies
Native-thread equivalence	10,003 records; 2/4/8 threads; 8/32/64 bands	Identical totals; unique native thread IDs
Validation	Malformed, missing or inconsistent metrics	Rejected; derived values recomputed
Production API	Valid / invalid / oversized requests	200 / 400 / 413
Cancellation	100,000 records; heavy computation	Workers stop; Run button re-enabled
Export	Production browser download	Valid measurement JSON saved
Responsive / accessibility	375 px viewport; automated axe audit	No horizontal overflow; 0 violations
Build / dependencies	Next production build; npm audit	Build passes; 0 vulnerabilities

Six automated test cases passed. The native-thread equivalence case checks nine thread/complexity combinations. Browser review also checked real completion, Gemini analysis and reduced-motion styling. Automated accessibility testing does not replace a full manual audit.

Run and demonstrate

Web: open the production URL, run the default experiment, explain the partition cards, compare elapsed times, export JSON, and analyze the result. Change the dataset size and thread count to show that more threads need not be faster.

Python: `python python/benchmark.py --records 10000 --threads 4 --bands 32`

Viva checkpoints

Explain a process versus a thread; why independent partitions avoid locks; how completion messages synchronize reduction; why one-worker and many-worker outputs must match; why startup, scheduling and the GIL can limit performance.

Included deliverables: source repository, source ZIP, this report, browser/Python measurement JSON, production screenshots, Gemini output, automated test log, dependency audit and HTTP checks.

References

MDN: Using Web Workers

Python: `concurrent.futures`

Python: threading and GIL considerations

Google: Gemini API documentation